

Computação Distribuída

Odorico Machado Mendizabal
Diogo Canut



Universidade Federal de Santa Catarina – UFSC
Departamento de Informática e Estatística – INE

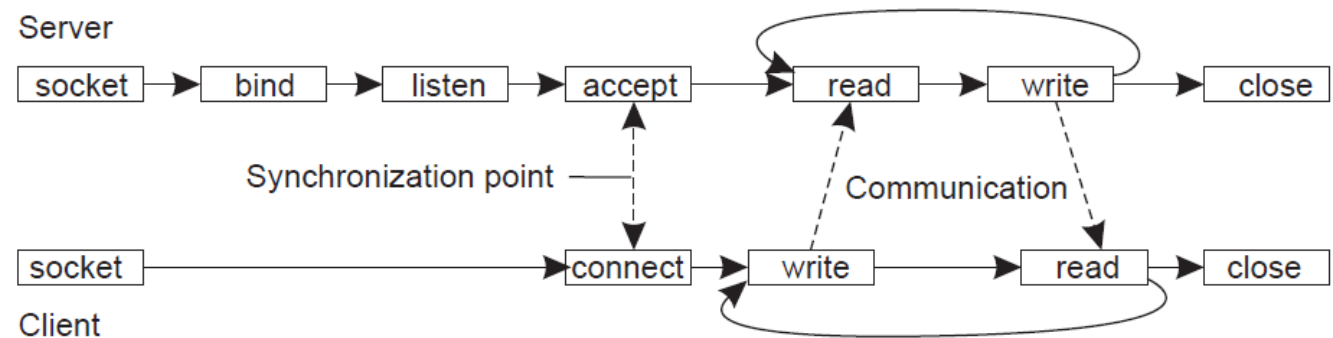


Middleware e serviços de mensagens

Comunicação Orientada a Mensagens

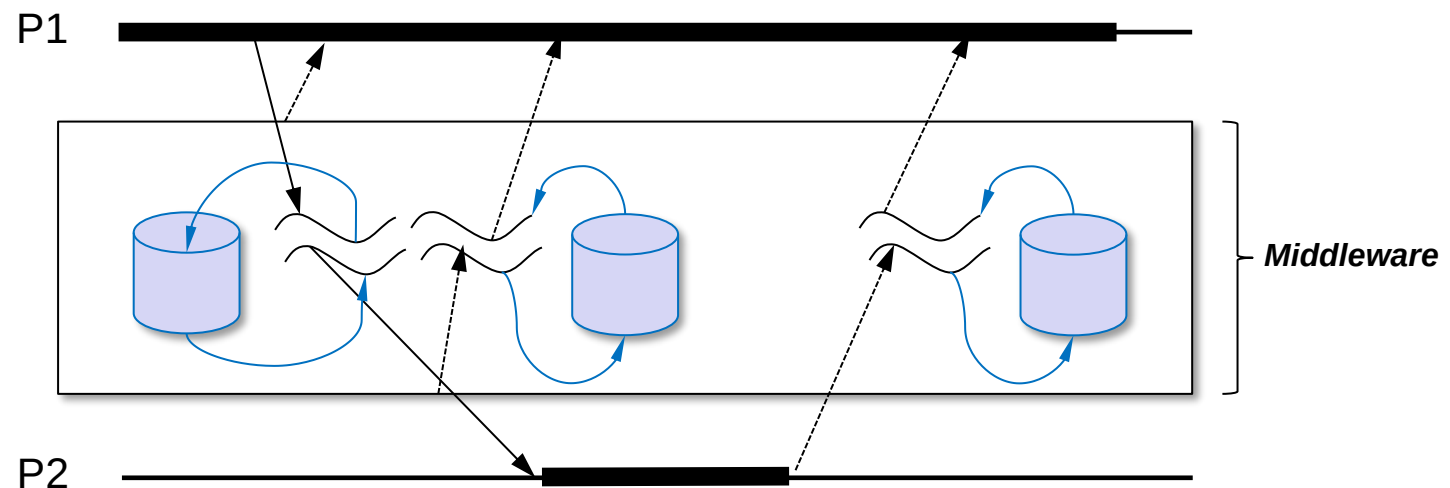
- Mensagens transientes

- Ex. Sockets e canais de comunicação



- Mensagens persistentes

- *Middleware* Orientado a Mensagens - MoM



Algumas Observações

Aplicações cliente/servidor normalmente baseiam-se em **comunicação transitiente**:

- Cliente e servidor devem estar ativos durante comunicação
- Cliente envia mensagem e **bloqueia** até receber resposta (não pode executar outra atividade enquanto espera)
- Servidor é responsável por atender e responder requisições
 - *Comportamento reativo, espera pelo recebimento de mensagens e responde*

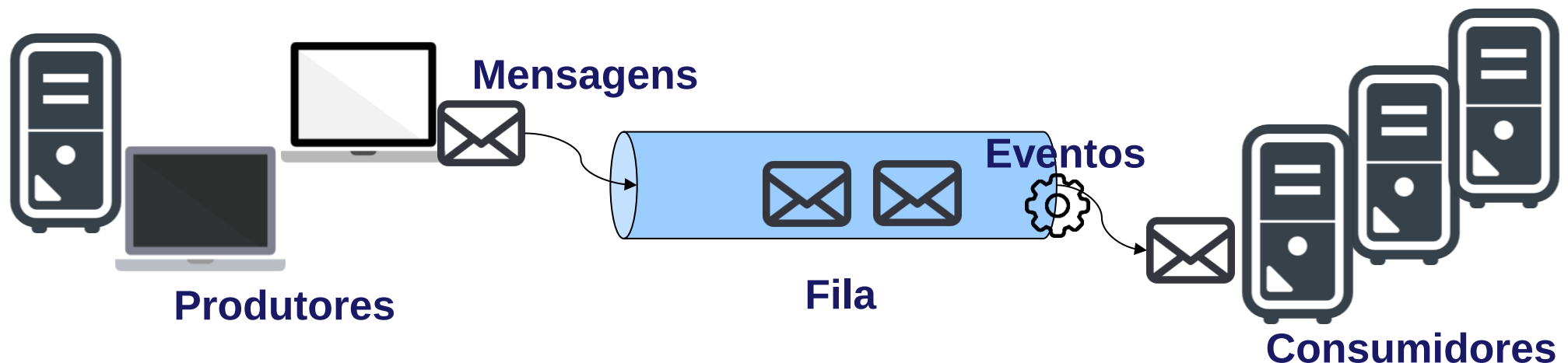
Modelo de **comunicação persistente**:

- Remetente não precisa esperar por respostas de requisições para prosseguir seu processamento
- Processos tornam-se mais autônomos (modelos diferentes do cliente/servidor)
- No modelo assíncrono com persistência, não há necessidade de ter os processos executando ao mesmo instante. Mensagens podem ser entregues posteriormente

Middleware Orientado a Mensagens - MoM

Filas de mensagens

- **Produtores:** Consiste em componentes que geram mensagens
- **Consumidores:** Se conectam à fila e obtêm as mensagens a serem processadas
- **Fila:** As mensagens colocadas na fila são armazenadas até que o consumidor as recupere
- **Evento:** Indicam a ocorrência de uma mudança de estado na aplicação. A publicação de uma mensagem é um evento



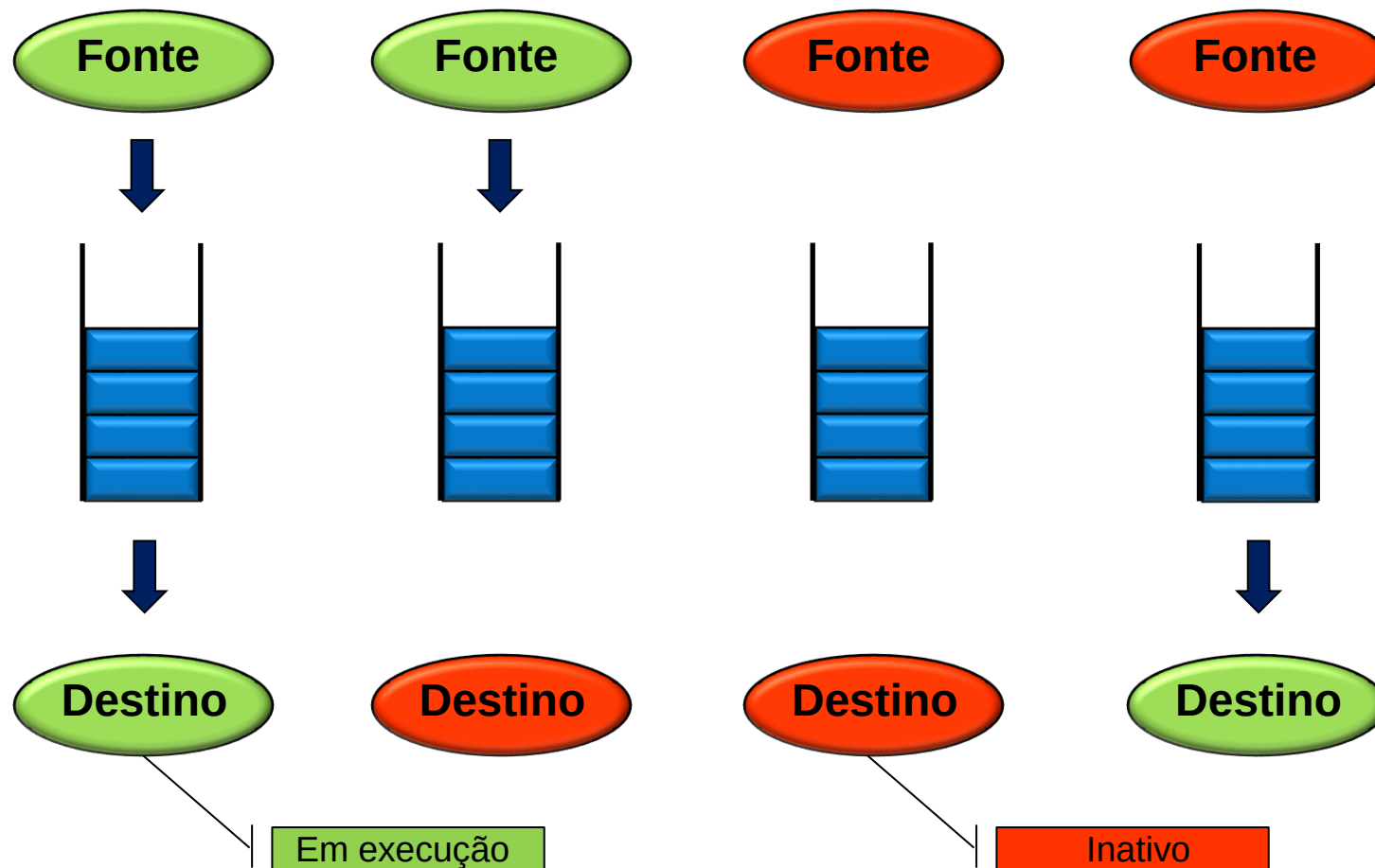
Middleware Orientado a Mensagens

Comunicação assíncrona persistente utilizando **Filas** implementadas pelo *middleware*

PUT	Adiciona uma mensagem a uma determinada fila
GET	Obtém uma mensagem de uma certa fila, bloqueando caso a mesma esteja vazia
POLL	Verifica a fila sem bloquear, obtendo uma mensagem caso a fila não esteja vazia
NOTIFY	Fornece um tratador (<i>handler</i>) para ser chamado quando uma mensagem for adicionada a fila

Filas de Mensagens

Estados possíveis durante a comunicação



Formato das Mensagens

Mensagens podem ter os mais diversos formatos

- *String*
- XML
- Binário

Cada Fila pode adotar um formato próprio

Regras de conversão podem ser aplicadas às mensagens por adaptadores (*adapters*) antes de serem colocadas na fila

Msg

```
.....&..2...t.CS&
.....[sACC..4...
.....4&.G.2.@...
&..2...@...&..2...
.t.CS&...].[s.CC
.....]_l...
.....6.....
.....P&.....u..
```

```
<?xml version="1.0" encoding="iso-8859-1" ?>
- <storedHU xmlns:aml="http://www.atlasti.com/hu/ns2003"
  lastSaved="2004-05-23T15:13:07" creator="ATLAS.ti"
  method="AML (ATLAS Markup Language)" version="WIN 5.0
  (Build 59)">
- <hermUnit name="The Sample" au="Admin" cDate="1991-03-
  11T13:22:37" mDate="2004-05-23T15:12:53" lastPD="5"
  prot="publicReadWrite">
  <comment>This HU is a toy example, but nevertheless
    draws some interesting relations between magic and
    religious terminology.</comment>
</hermUnit>
- <coAuthors>
  <coAuthor name="ADMIN" />
  <coAuthor name="ANDREAS" />
  <coAuthor name="ATLAS" />
  <coAuthor name="JOOP" />
  <coAuthor name="TM" />
</coAuthors>
- <codes size="52">
- <code name="A Formula" id="co_1" au="Thomas M"
  cDate="2003-03-04T14:30:57" mDate="2003-03-
  07T13:19:42" cCount="0" qCount="1">
```

```
nome: Fabricio Mauricio
id: 263756
sexo: m
idade: 33
```

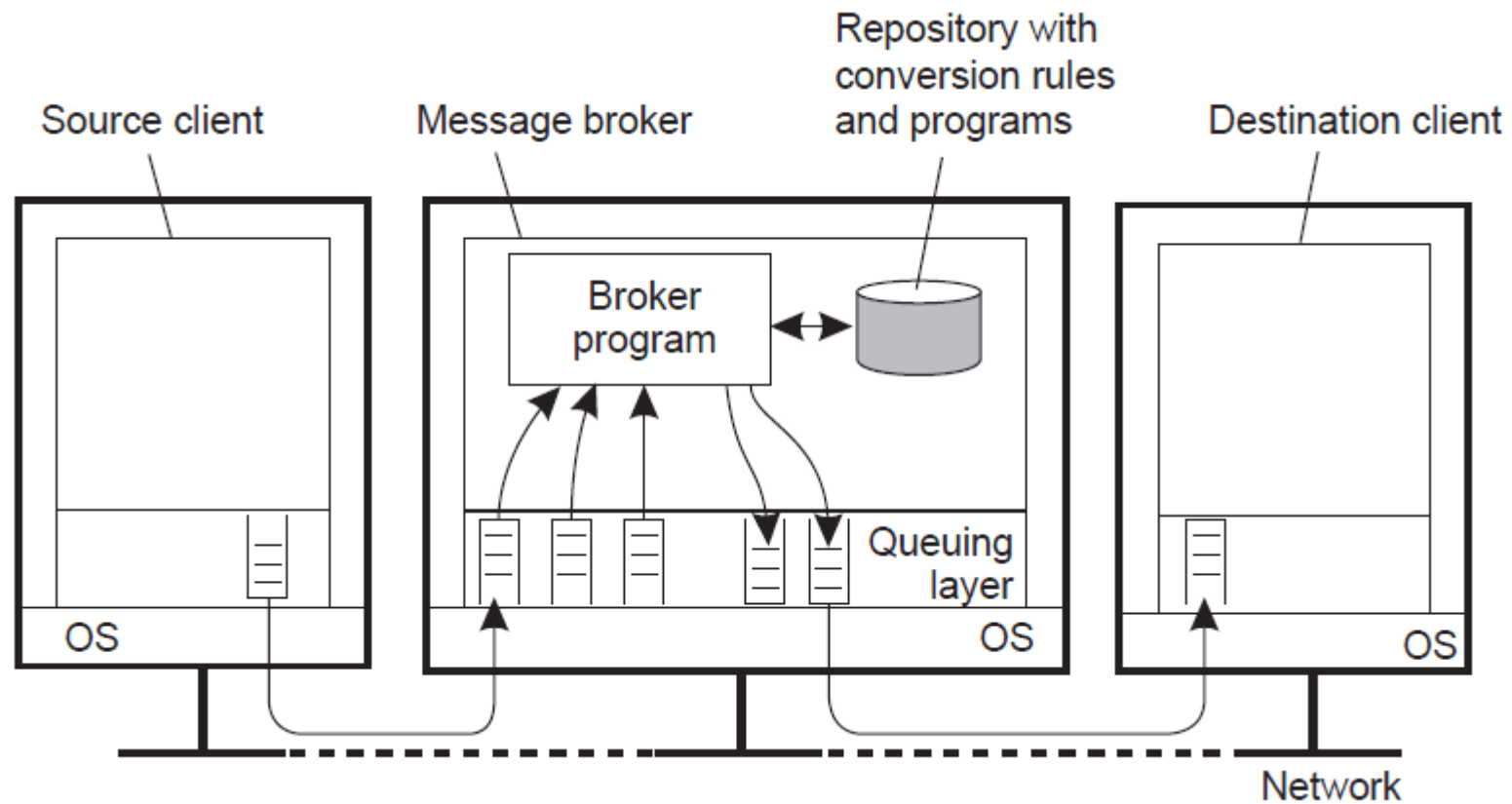
Message Broker

Sistemas de Filas de Mensagens adotam um **protocolo de troca de mensagens comum**. Todas aplicações utilizam o mesmo formato de mensagens (estrutura e representação de dados)

Message Broker (Agente)

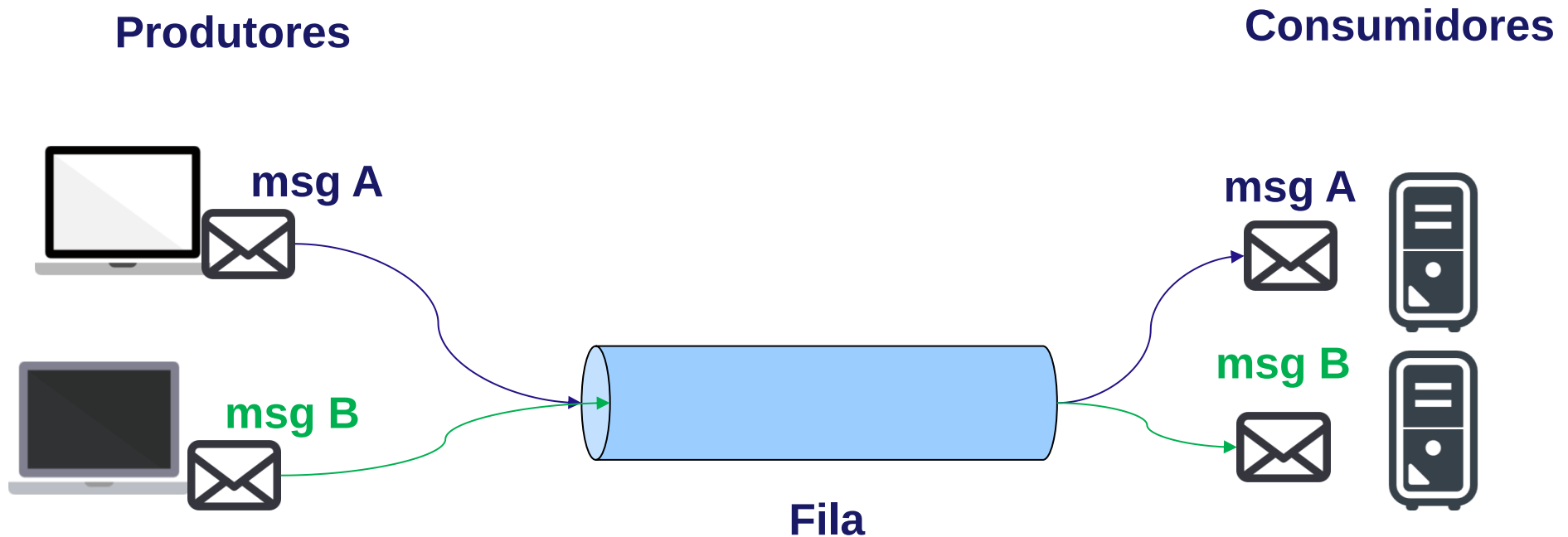
- Componente centralizado que permite a interoperabilidade em um sistema de Filas de Mensagem (MQ – *Message-Queuing*) - trata da heterogeneidade
- Transforma mensagens recebidas no formato alvo
- Pode atuar como *Gateway* de aplicação

Message Broker



Filas ponto a ponto

- Implementam um modelo tradicional produtor/consumidor.
- Produtores geram mensagens, que são persistidas na fila
- Consumidores competem pelo acesso à fila, removendo as mensagens em ordem FIFO
- Cada mensagem é processada uma vez por algum consumidor



Modelo *Publish/Subscribe*

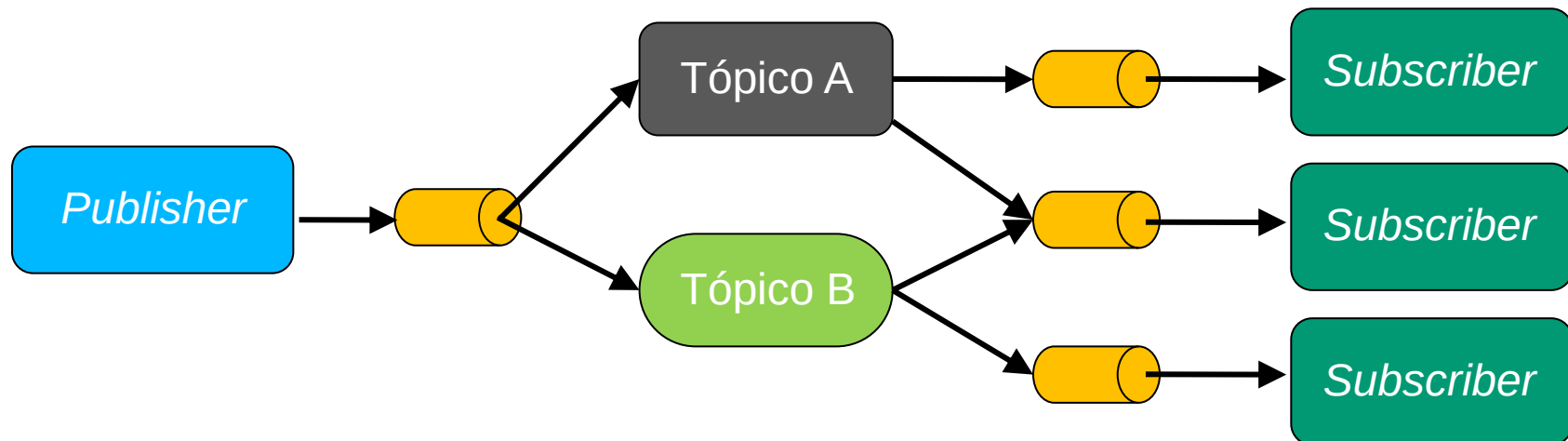
- Processos registram interesse em receber mensagens (*inscrição em um tópico*) e/ou publicar mensagens (*publicação em um tópico*)
 - Filtragem de assuntos
- Diferentes níveis
 - Aplicação final: *newsgroups*
 - Primitivas de comunicação
- Modelos de comunicação
 - *push* e *pull*
 - Similar a sistemas baseados em eventos

Modelo *Publish/Subscribe*

Implementado na maioria dos *Message Brokers*

Funcionamento:

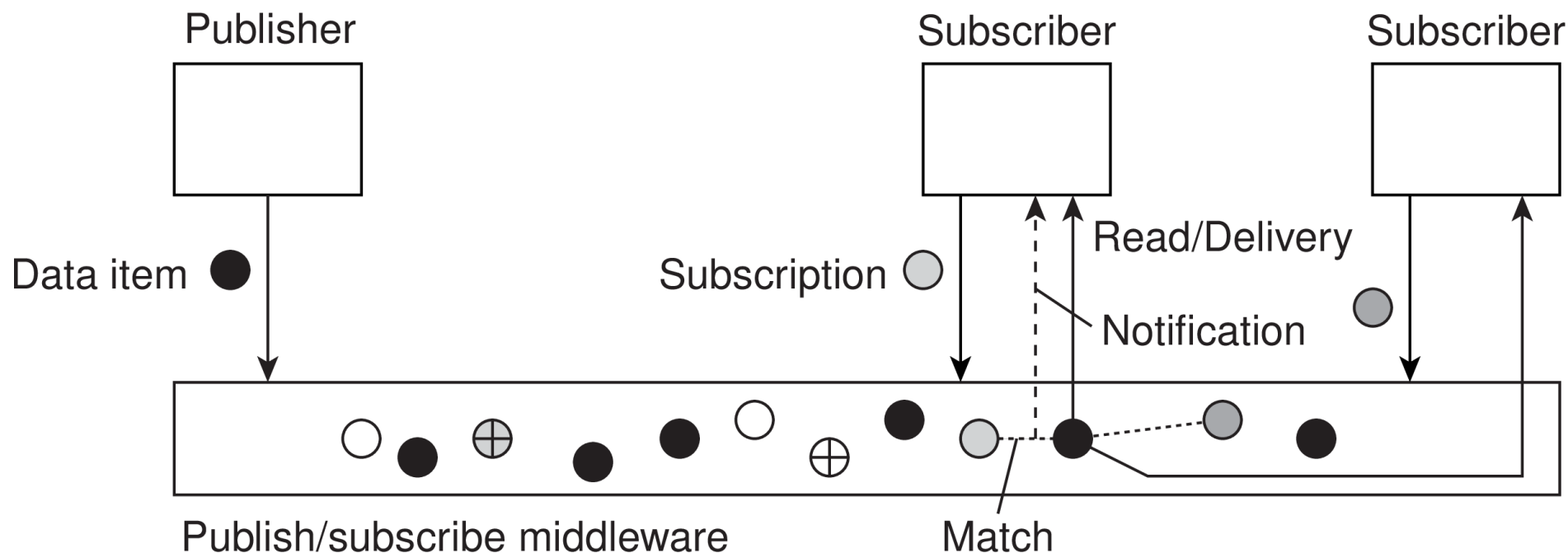
- *Publisher*: produtor de conteúdo a ser consumido
- *Subscriber*: “assina” um conjunto de conteúdos de seu interesse



Exemplo: Dropbox

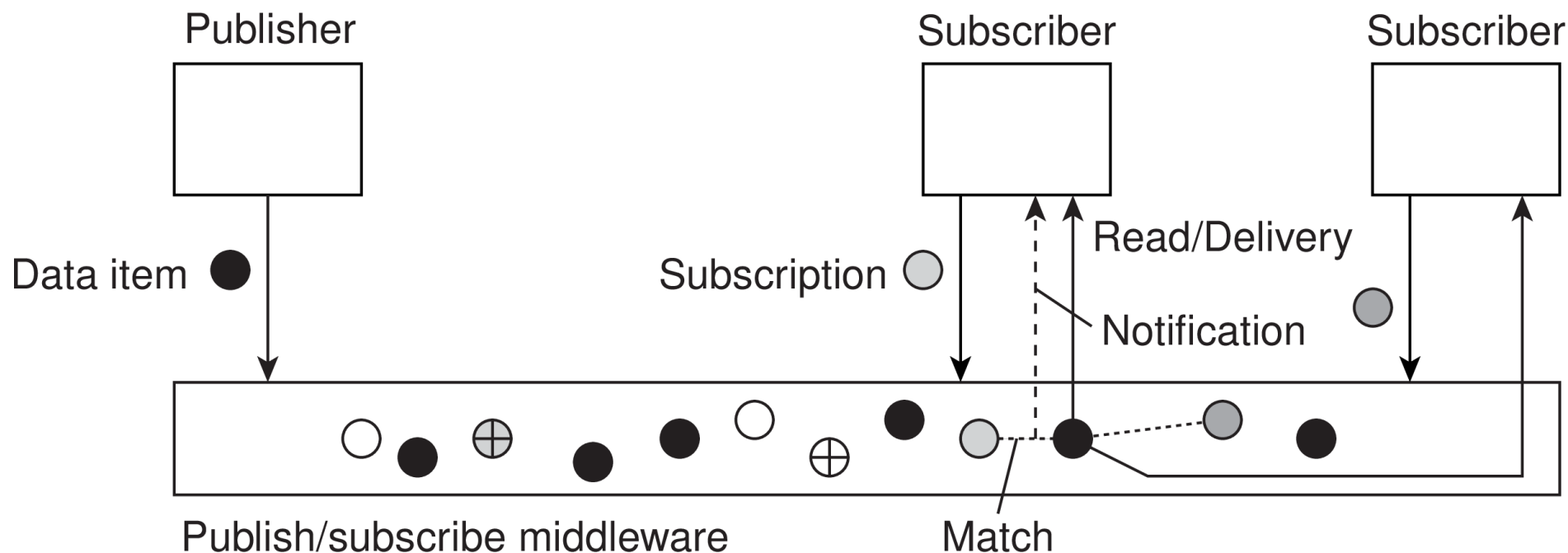
Modelo *Publish/Subscribe* – *matching*

- Como determinar a correspondência entre itens e inscrições?
 - Eventos podem ser descritos como pares (*atributo*, *valor*)
 - Inscrições baseadas em tópicos:
 - Correspondência se dá: *atributo* = *valor*
 - Inscrições baseadas conteúdo:
 - Correspondência se dá: *atributo* $\in$ *intervalo*



Modelo *Publish/Subscribe* – *matching*

- Exemplo
 msg(atributo: “INE5418”, valor: “mudança da data da prova”)
- Inscrição por tópico:
 - Tópico: INE5418
- Inscrição por conteúdo:
 - Conteúdo: Disciplinas de CCO (INE5401, INE5402, ... INE5418, ...)

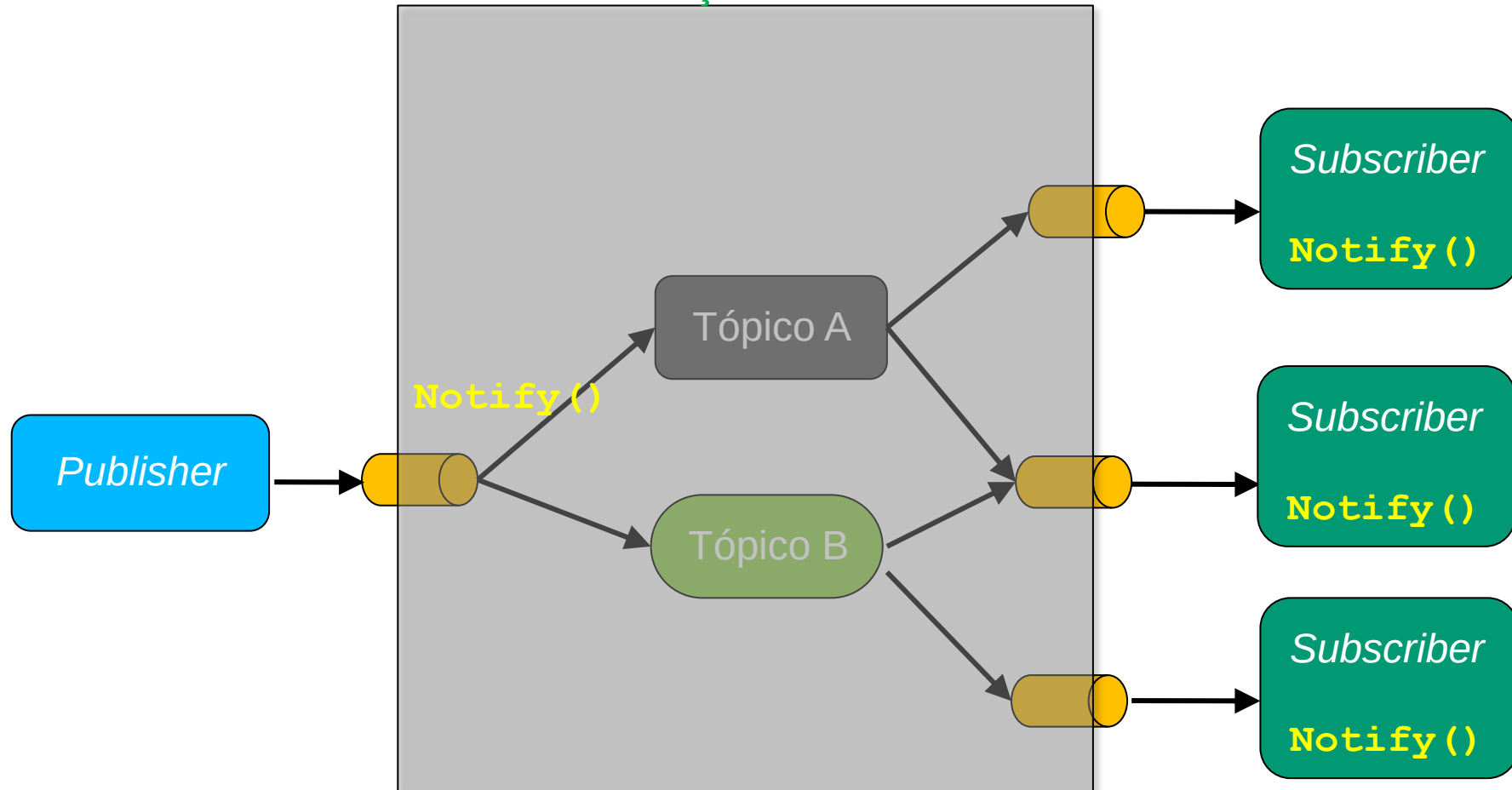


Modelo *Publish/Subscribe* – Padrão de Interação

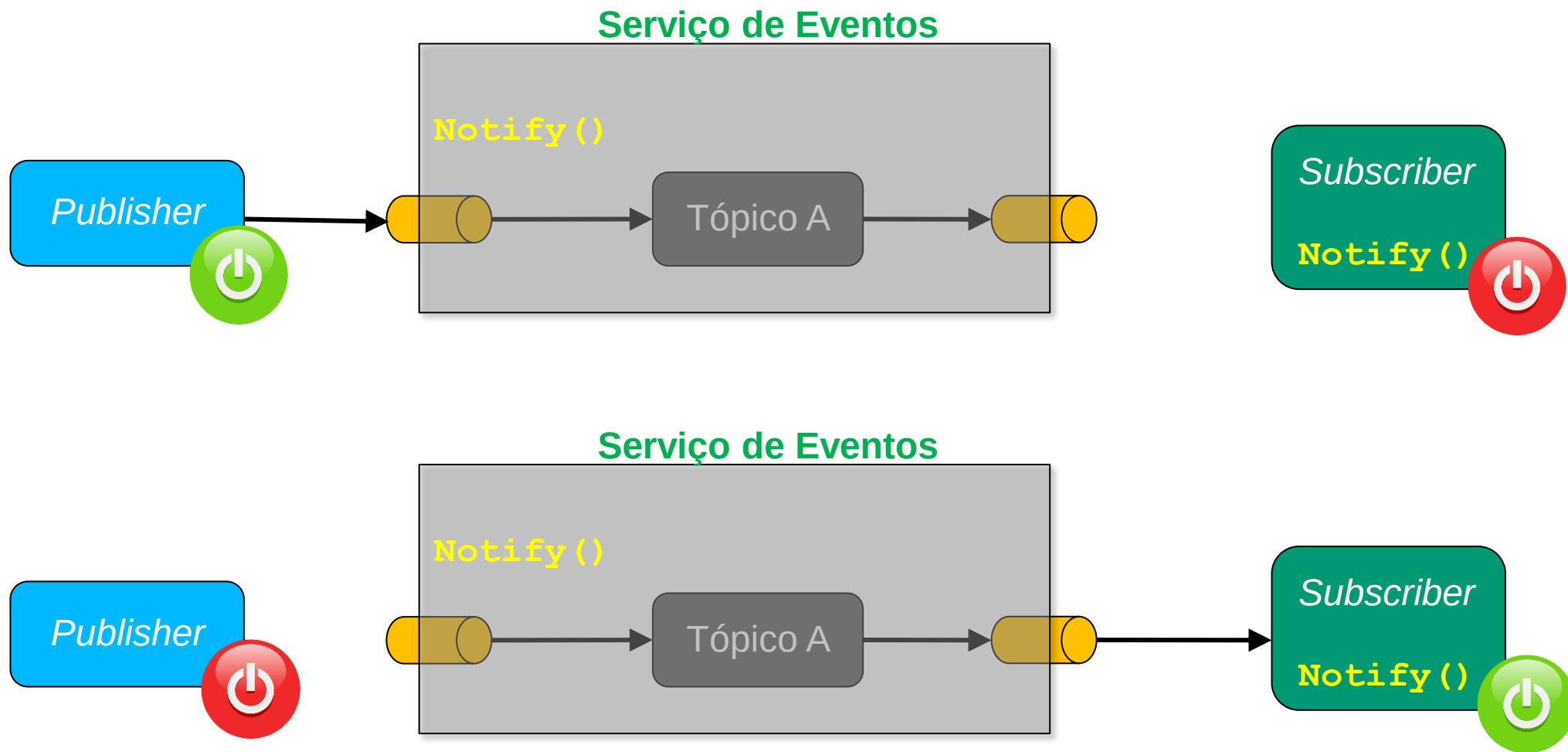
- Assinantes registram interesse em eventos ou padrões de eventos
- publicação de evento gera notificação assíncrona
- Desacoplamento
 - tempo
 - espaço
 - sincronização

Desacoplamento Espacial

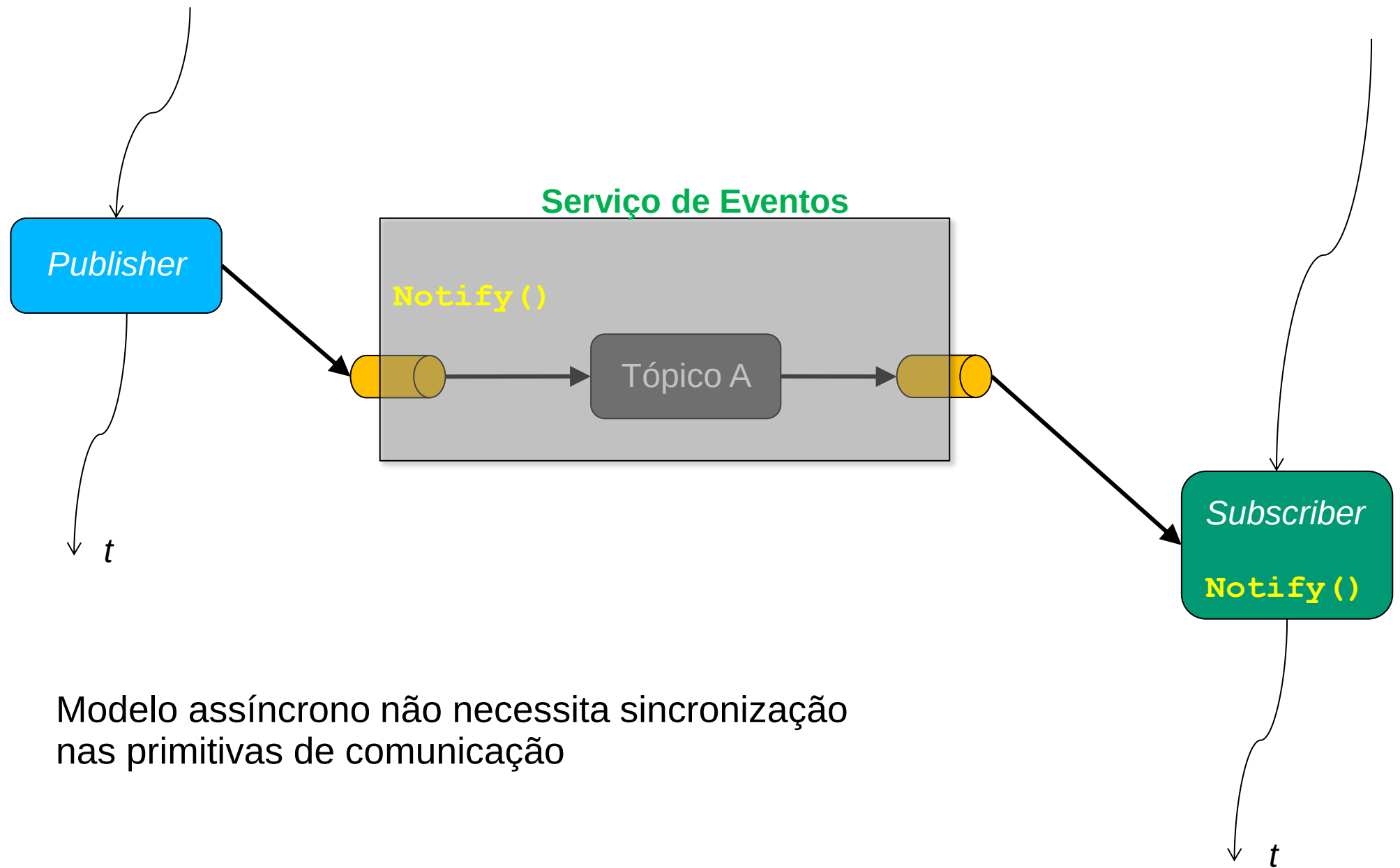
Serviço de Eventos



Desacoplamento Temporal



Desacoplamento de Sincronização



Vantagens dos Sistemas MQ

- **Heterogeneidade**: Diferentes aplicações conseguem trocar mensagens entre si, independentemente da linguagem de programação, SO, HW
- **Sistemas assíncronos**, mensagens podem ser enviadas sem que cliente fique bloqueado esperando confirmação
- **Persistência** de mensagens: Mensagens enviadas são mantidas até que não sejam mais necessárias (replicação pode ser usada de modo a adicionar tolerância a falhas e aumentar disponibilidade das Filas)
- Múltiplos receptores podem consumir mensagens em uma fila sem que haja coordenação e sincronização entre eles (modelo *Publisher/Subscriber*)

Padrões de Mercado

- JMS – Java Message Service
- IBM Websphere MQ
- Apache ActiveMQ
- BEA Weblogic JMS
- JBoss Messaging
- Microsoft Message Queue Server (MSMQ)
- RabbitMQ
- Zero MQ
- Nano MSG

Leituras Adicionais

[1] P. Eugster, P. Ferrer, R. Guerraoui e A. Kermarrec. The many faces of publish/subscribe. ACM Computing Surveys, 35(2), jun 2003.

[2] S. Kamburugamuve, G. Fox. Survey of Distributed Stream Processing. 2016

Survey of Distributed Stream Processing

Supun Kamburugamuve, Geoffrey Fox
School of Informatics and Computing
Indiana University, Bloomington, IN, USA

Applications in which large amounts of data generated in external environments are pushed to servers for processing. These applications include sensor-based monitoring, stock trading, web traffic processing, network analytics. The data generated by these applications can be seen as streams of events or tuples. In these systems, the information can no longer be processed in real time by the systems. Instead, the information is pushed to the system as unbounded sequences of event tuples. In these systems called distributed stream processing frameworks (DSSPs), the information is processed in batch processing. For the past few years, batch processing has been the dominant paradigm for data processing. This included efforts to make batch processing more efficient. People have realized that batch processing is not well adapted to the loosely coupled applications, the publish/subscribe paradigm, where subscribers register their interest in data and receive updates asynchronously. Well adapted to the loosely coupled applications, the publish/subscribe paradigm, where subscribers register their interest in data and receive updates asynchronously.

The Many Faces of Publish/Subscribe

PATRICK TH. EUGSTER
Swiss Federal Institute of Technology, Lausanne

PASCAL A. FELBER
Institut Eurécom

RACHID GUERRAOUI
Swiss Federal Institute of Technology, Lausanne

AND
ANNE-MARIE KERMARREC
Microsoft Research

Well adapted to the loosely coupled applications, the publish/subscribe paradigm, where subscribers register their interest in data and receive updates asynchronously. Well adapted to the loosely coupled applications, the publish/subscribe paradigm, where subscribers register their interest in data and receive updates asynchronously.

Referências

Parte destes slides são baseadas em material de aula dos livros:

- *Coulouris, George; Dollimore, Jean; Kindberg, Tim; Blair, Gordon. Sistemas Distribuídos: Conceitos e Projetos. Bookman; 5ª edição. 2013. ISBN: 8582600534*
- *Tanenbaum, Andrew S.; Van Steen, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2007. Pearson Universidades; 2ª edição. ISBN: 8576051427*

